Day 7 — Week 1 Consolidation and Strict Review

Revision summary

Week 1 should leave you able to build and defend a small production-style analytics service—not merely explain Python, FastAPI, PostgreSQL, Redis, testing, or statistics independently.

The strongest interview signal is this end-to-end chain:

> Validated input → idempotent ingestion → transactional persistence → analytical SQL → stable API contract → safe caching → observable execution → deterministic tests → business explanation.

The largest risk is claiming the PoC is production-ready without demonstrating clean installation, duplicate-request handling, deterministic pagination, rollback behavior, and meaningful tests.

Priority table

Priority	Area	What must be demonstrated
P0	Reproducibility	A new reviewer can clone, start, migrate, seed, test, and call the API without undocumented steps
P0	Correctness	Variance calculations, filtering, aggregation grain, zero-budget handling, and duplicate ingestion are correct
P0	API contracts	Validation, error envelope, idempotency, pagination, status codes, and response schemas are explicit
P1	SQL reasoning	You can explain joins, CTEs, window functions, indexes, and query-plan trade-offs
P1	Reliability	Transaction boundaries, retry classification, timeouts, cancellation, and cache failure behavior are defensible
P1	Testing	Tests cover behavior and failure modes, not only happy-path lines
P1	Communication	Architecture and statistical conclusions can be explained without implementation-level noise
P2	Coding speed	A medium problem can be solved correctly within 25–30 minutes

---

Part A — Closed-book assessment

Do not read the answer section until you have completed the quiz and exercises.

1. Twenty-question closed-book quiz

Recommended time: **25 minutes**.

Write answers in one to three sentences each.

Python architecture and FastAPI

1. Why should a FastAPI route normally delegate business logic to a service rather than calling PostgreSQL and Redis directly?
2. What is wrong with declaring an endpoint `async def` and then calling a synchronous database or HTTP client inside it?
3. Give one situation where a synchronous FastAPI endpoint is preferable to an asynchronous one.
4. A request passes JSON parsing but violates a business rule such as `end_period < start_period`. Which response status would you normally return, and why?
5. What is the difference between authentication, authorization, and input validation?

Contracts, ingestion, and pagination

1. What guarantee should an idempotency key provide?
2. A client reuses an idempotency key with a different request body. What should the service do?
3. Why is `OFFSET 100000 LIMIT 50` unsuitable for a large, frequently updated table?
4. What properties must the ordering columns of a cursor-paginated query have?

PostgreSQL and analytical SQL

1. What problem does a transaction solve during multi-row financial ingestion?
2. When would a window function be preferable to a `GROUP BY`?
3. A query filters by `period`, groups by `department_id`, and orders by unfavorable variance. What information would you inspect before creating an index?
4. Why can joining budget rows directly to individual expense rows inflate the budget total?

Redis and asynchronous reliability

1. What is a cache stampede?
2. Should an API normally fail if Redis is unavailable but PostgreSQL is healthy? State the trade-off.
3. Which failures are generally retryable: validation failure, timeout, deadlock, authentication failure, or connection reset?
4. Why should retry logic include exponential backoff and jitter?

Testing, logging, and statistics

1. What is the difference between a unit test and an integration test for a repository?
2. What does a 95% confidence interval mean, and what does it not mean?
3. A result is statistically significant but saves only ₹500 per month for a large business. What further question must be answered?

---

2. Thirty-minute backend/API design mock

Case

Design a finance analytics service that:

- Ingests budget and expense records.
- Supports monthly budget-versus-actual summaries.
- Identifies high-value expense exceptions.
- Provides department, cost-centre, vendor, and transaction drill-down.
- Receives duplicate submissions from upstream systems.
- Serves up to millions of expense rows.
- Must remain usable when Redis is temporarily unavailable.
- Must provide an audit trail suitable for financial investigation.

Time allocation

Minutes	Expected activity
0–3	Clarify requirements and state assumptions
3–7	Define APIs and contracts
7–12	Define data model and constraints
12–17	Explain ingestion and idempotency
17–21	Explain analytical queries and indexes
21–24	Explain caching
24–27	Explain reliability, security, and observability
27–30	Explain tests and trade-offs

Questions the interviewer will interrupt with

1. How do you prevent the same expense from being counted twice?
2. What happens when two identical ingestion requests arrive concurrently?
3. How do you paginate while new expenses are being inserted?
4. What is cached, and how is it invalidated?
5. Why not store everything in Redis?
6. How do you reconcile late-arriving expenses?
7. How do you prove that the summary is traceable to source transactions?
8. What happens after PostgreSQL commits but cache invalidation fails?
9. Which API operations can safely be retried?
10. How would the design change if ingestion became ten times larger?

Design scoring rubric

Area	Points
Requirements and assumptions	4
API contracts	6

Data model and constraints	6
Idempotency and transactions	6
SQL and indexing	5
Caching and consistency	4
Reliability and observability	5
Testing and trade-offs	4
Total	**40**

A senior-level answer should reach at least **30/40** without requiring repeated interviewer hints.

---

3. Twenty-minute analytical SQL mock

Assume PostgreSQL.

```
CREATE TABLE budgets (
    budget_id       BIGINT PRIMARY KEY,
    period          DATE NOT NULL,
    department_id   BIGINT NOT NULL,
    cost_center_id  BIGINT NOT NULL,
    amount           NUMERIC(18, 2) NOT NULL,
    version          INTEGER NOT NULL,
    updated_at       TIMESTAMPTZ NOT NULL
);

CREATE TABLE expenses (
    expense_id       BIGINT PRIMARY KEY,
    source_record_id TEXT NOT NULL UNIQUE,
    period           DATE NOT NULL,
    department_id    BIGINT NOT NULL,
    cost_center_id   BIGINT NOT NULL,
    vendor_id        BIGINT NOT NULL,
    amount            NUMERIC(18, 2) NOT NULL,
    approval_status  TEXT NOT NULL,
    posted_at         TIMESTAMPTZ NOT NULL
);
```

Task

For January through June 2026:

1. Select only the latest budget version for each period, department, and cost centre.
2. Aggregate only approved expenses.
3. Preserve combinations that have only a budget or only actual expenses.
4. Calculate:
    - budget,
    - actual,

    - variance as `actual - budget`,
    - variance percentage.
5. Return the top three unfavourable cost centres for every department and period.
6. Make ranking deterministic when variances are tied.

Explain:

- Why pre-aggregation is necessary.
- How zero budgets are handled.
- Which indexes you would evaluate.

---

4. Statistical communication exercise

Recommended preparation time: **3 minutes**. Speaking limit: **2 minutes**.

Result

During a six-month analysis, a department exceeded its monthly budget by an estimated average of **₹18 lakh**. The 95% confidence interval is **₹12 lakh to ₹24 lakh**, and the hypothesis test produced `p < 0.01`.

Explain this result to a finance executive. Your explanation must include:

- The likely business conclusion.
- What uncertainty remains.
- What the p-value does not prove.
- One recommended action.
- One limitation that should be checked.

Do not define statistical terminology unless necessary.

---

5. Timed DSA exercise

Recommended time: **25 minutes**.

Problem: Longest substring without repeating characters

Given a string `s`, return the length of its longest substring containing no repeated characters.

```
Input:  "abcabcbb"
Output: 3
Explanation: "abc"
```

```
Input:  "bbbbb"
Output: 1
```

```
Input:  "pwwkew"
Output: 3
Explanation: "wke"
```

Before coding, write

1. Recognition signals.
2. Brute-force approach.
3. Optimized invariant.
4. Edge cases.
5. Time and space complexity.

Stop here until the assessment is complete

Part B — Strict review and answer key

6. Closed-book quiz answers

1. **Separation of concerns.** The route should handle HTTP concerns, while the service owns business rules and orchestration. This makes the logic reusable and independently testable.
2. A synchronous call blocks the event-loop thread, preventing other coroutines from progressing. Use an async client, execute the blocking work in a thread pool, or keep the endpoint synchronous.
3. Use a synchronous endpoint when the work depends primarily on blocking libraries and there is no useful asynchronous concurrency.
4. Usually `422 Unprocessable Entity`, because the syntax is valid but the supplied values violate request semantics. A consistent domain-validation policy matters more than debating `400` versus `422`.
5. Authentication establishes identity, authorization checks permission, and validation checks whether the supplied data is acceptable.
6. Repeating the same logical request with the same key and payload must not create an additional side effect; it should return the original result or an equivalent result.
7. Reject it, normally with `409 Conflict`. Silent reuse could associate one key with two different operations.
8. PostgreSQL must scan and discard many preceding rows. Concurrent inserts or deletes can also cause skipped or repeated results between pages.
9. Ordering must be deterministic and backed by a unique tie-breaker, such as `(posted_at, expense_id)`. Cursor predicates must use the same ordering.
10. It ensures all related changes succeed or fail together, preventing partial ingestion and internally inconsistent financial data.
11. Use a window function when row-level detail must be retained while calculating rank, running totals, lag values, or partition-level aggregates.

12. Inspect row counts, selectivity, existing indexes, join predicates, sort cost, grouping cost, execution plans, data distribution, and actual query frequency.
13. Each budget row is repeated once per matching expense. Aggregate each side to the same grain before joining.
14. Many requests simultaneously miss the same cache entry and all execute the expensive underlying computation.
15. Usually no. The service should fall back to PostgreSQL, accepting higher latency and database load. This assumes the database has sufficient protection against overload.
16. Timeouts, deadlocks, transient connection resets, and some connection failures can be retryable. Validation and authentication failures are not retryable without changing the request or credentials.
17. Backoff prevents immediate repeated pressure; jitter prevents many clients from retrying simultaneously.
18. A unit test isolates repository logic or its callers with fakes or mocks. An integration test executes against a real PostgreSQL instance and verifies SQL, constraints, transactions, and type mapping.
19. Under repeated equivalent sampling, 95% of intervals produced by the method would contain the true parameter. It does not mean there is a 95% probability that the already-calculated fixed interval contains it.
20. Determine whether the effect is economically meaningful after implementation cost, operational risk, and opportunity cost.

Quiz evaluation

Correct answers	Assessment
18–20	Strong Week 1 recall
15–17	Interview-capable, with targeted gaps
12–14	Knowledge exists but retrieval is unreliable
8–11	Fundamentals are fragmented
0–7	Repeat core Week 1 exercises before adding complexity

Do not award full credit for vague answers containing only keywords.

---

7. Backend design review

Strong answer structure

Requirements and assumptions

State these before drawing components:

- Budget and expense records have stable upstream identifiers.
- Approved expenses contribute to official actuals.
- Late-arriving and corrected data are expected.
- PostgreSQL is the system of record.

- Redis is an optimization, not a source of truth.
- Audit records are append-only.
- Financial amounts use decimal database types, not floating point.
- All stored timestamps use UTC; finance periods follow an explicit business timezone.

API surface

```
POST /v1/ingestion/budgets
POST /v1/ingestion/expenses

GET /v1/variance-summary
GET /v1/exceptions
GET /v1/trends
GET /v1/expenses
GET /v1/expenses/{expense_id}
GET /v1/ingestion/{request_id}
```

Important contract elements:

- `Idempotency-Key` on ingestion operations.
- Explicit period and scope filters.
- Maximum page size.
- Cursor rather than large offsets.
- Stable sort specification.
- Consistent error envelope.
- Correlation ID in request and response.
- Separate summary and drill-down response schemas.

Data model

Core tables:

```
departments
cost_centres
vendors
budget_versions
expenses
ingestion_requests
approval_events
audit_events
```

Critical constraints:

- Unique upstream `source_record_id`.
- Unique budget version or source version identifier.
- Foreign keys to department, cost centre, and vendor.
- Check constraints for valid statuses and amounts where appropriate.
- Unique idempotency key.
- Immutable audit-event identity.

- Version or timestamp for optimistic concurrency when records can be amended.

Ingestion path

```
Request
  │
  ├─ Validate envelope and row limits
  ├─ Canonicalize payload and calculate hash
  ├─ Atomically claim idempotency key
  ├─ Validate domain references
  ├─ Upsert using stable source identifiers
  ├─ Write audit event
  ├─ Commit PostgreSQL transaction
  └─ Invalidate or version affected cache keys
```

Concurrent identical requests must result in only one committing the business mutation. The other request should replay the stored result or observe an in-progress state.

SQL and indexing

Aggregate budgets and actuals independently before joining. Candidate indexes must be justified through actual query plans.

Likely indexes to evaluate:

```
CREATE INDEX expenses_summary_idx
    ON expenses (period, department_id, cost_center_id)
    INCLUDE (amount)
    WHERE approval_status = 'APPROVED';

CREATE INDEX expenses_vendor_drilldown_idx
    ON expenses (vendor_id, posted_at DESC, expense_id DESC);

CREATE INDEX budgets_latest_idx
    ON budgets (
        period,
        department_id,
        cost_center_id,
        version DESC,
        updated_at DESC
    );
```

These are candidates, not automatic recommendations. Write amplification and table size must be considered.

Caching

Cache aggregate responses, not raw transactional truth.

A suitable key contains:

```
schema version
data version
```

```
endpoint
period range
department
cost centre
sort/filter parameters
```

A namespace or dataset-version strategy is safer than attempting to enumerate every affected cache key.

When Redis fails:

- Log the cache failure.
- Read from PostgreSQL.
- Apply database concurrency limits.
- Do not return incorrect stale data merely to preserve latency.

Failure handling

- Retry only transient failures.
- Do not blindly retry financial writes unless idempotency is guaranteed.
- Use a total request timeout budget.
- Set database statement timeouts.
- Bound ingestion batch size and concurrency.
- Roll back the complete transaction on row failure unless partial acceptance is an explicit contract.
- Treat cache invalidation after commit as eventually consistent and recover using short TTLs or versioned keys.

Observability

Each log should expose fields such as:

```
event
correlation_id
request_id
idempotency_key_hash
endpoint
duration_ms
row_count
cache_status
database_operation
error_code
```

Do not log raw bank details, authorization tokens, full payloads, or unnecessary vendor information.

Testing strategy

- Domain-unit tests.
- FastAPI contract tests.
- PostgreSQL integration tests.

- Redis fallback tests.
- Concurrency tests for idempotency.
- Migration tests.
- Deterministic data-generation tests.
- End-to-end clean-environment smoke test.

Common Google-style rejection reasons

- Saying “microservices” before establishing scale or ownership boundaries.
- Using Redis as the source of truth.
- Claiming exactly-once delivery without defining its scope.
- Using an idempotency key without a payload hash.
- Using offset pagination while claiming stable pagination.
- Applying async everywhere without identifying blocking operations.
- Listing indexes without discussing query plans or write cost.
- Returning cached financial data without a freshness contract.
- Logging full request bodies.
- Ignoring late data, corrections, and auditability.

---

8. Analytical SQL solution

```
WITH latest_budget_rows AS (
    SELECT
        period,
        department_id,
        cost_center_id,
        amount,
        ROW_NUMBER() OVER (
            PARTITION BY period, department_id, cost_center_id
            ORDER BY version DESC, updated_at DESC, budget_id DESC
        ) AS row_num
    FROM budgets
    WHERE period >= DATE '2026-01-01'
      AND period <  DATE '2026-07-01'
),
budget_agg AS (
    SELECT
        period,
        department_id,
        cost_center_id,
        SUM(amount) AS budget
    FROM latest_budget_rows
    WHERE row_num = 1
    GROUP BY period, department_id, cost_center_id
),
actual_agg AS (
    SELECT
        period,
        department_id,
```

```
        cost_center_id,
        SUM(amount) AS actual
    FROM expenses
    WHERE period >= DATE '2026-01-01'
      AND period <  DATE '2026-07-01'
      AND approval_status = 'APPROVED'
    GROUP BY period, department_id, cost_center_id
),
combined AS (
    SELECT
        COALESCE(b.period, a.period) AS period,
        COALESCE(b.department_id, a.department_id) AS department_id,
        COALESCE(b.cost_center_id, a.cost_center_id) AS cost_center_id,
        COALESCE(b.budget, 0::NUMERIC) AS budget,
        COALESCE(a.actual, 0::NUMERIC) AS actual
    FROM budget_agg AS b
    FULL OUTER JOIN actual_agg AS a
      ON a.period = b.period
     AND a.department_id = b.department_id
     AND a.cost_center_id = b.cost_center_id
),
calculated AS (
    SELECT
        period,
        department_id,
        cost_center_id,
        budget,
        actual,
        actual - budget AS variance,
        100.0 * (actual - budget) / NULLIF(budget, 0) AS variance_pct
    FROM combined
),
ranked AS (
    SELECT
        period,
        department_id,
        cost_center_id,
        budget,
        actual,
        variance,
        variance_pct,
        ROW_NUMBER() OVER (
            PARTITION BY period, department_id
            ORDER BY variance DESC, cost_center_id ASC
        ) AS exception_rank
    FROM calculated
    WHERE variance > 0
)
SELECT
    period,
    department_id,
    cost_center_id,
    budget,
    actual,
```

```
    variance,
    variance_pct,
    exception_rank
FROM ranked
WHERE exception_rank <= 3
ORDER BY
    period,
    department_id,
    exception_rank,
    cost_center_id;
```

Concise explanation

- **Pre-aggregation:** prevents a budget amount from being duplicated once for every matching expense.
- **Latest version:** `ROW_NUMBER` chooses one deterministic budget record at each analytical grain.
- **Missing rows:** `FULL OUTER JOIN` retains budget-only and actual-only combinations.
- **Zero budget:** `NULLIF(budget, 0)` returns a null percentage instead of raising division-by-zero or inventing a misleading percentage.
- **Deterministic ranking:** `cost_center_id` breaks equal-variance ties.

Indexes to evaluate

```
CREATE INDEX budgets_period_grain_version_idx
    ON budgets (
        period,
        department_id,
        cost_center_id,
        version DESC,
        updated_at DESC,
        budget_id DESC
    );

CREATE INDEX expenses_approved_summary_idx
    ON expenses (period, department_id, cost_center_id)
    INCLUDE (amount)
    WHERE approval_status = 'APPROVED';
```

Verify them using `EXPLAIN (ANALYZE, BUFFERS)`. Do not claim improvement without observing the plan and runtime on representative data.

---

9. Executive explanation of the statistical result

A strong answer should sound like this:

> The analysis indicates that this department is consistently overspending rather than showing only random month-to-month variation. Our best estimate is an average monthly overspend of ₹18 lakh, with a plausible range of approximately ₹12 lakh to ₹24 lakh under the assumptions of the analysis. The low p-value means the observed pattern would be

> unusual if the department had no real average overspend, but it does not prove the cause or guarantee that the same amount will continue. I recommend identifying the vendors and cost centres responsible for the excess and checking whether the pattern remains after adjusting for seasonality, one-time purchases, and changes in business volume.

Strict communication review

Reject or correct these statements:

- “There is a 95% probability that the true value lies in this interval.”
- “The p-value proves the department is overspending.”
- “Statistical significance proves the amount is material.”
- “The department will overspend by exactly ₹18 lakh next month.”
- “We should immediately reduce the department’s budget.”

The executive needs the decision implication, uncertainty, likely drivers, and next action—not a statistics lecture.

---

10. Timed DSA review

Recognition signals

- The problem asks about a contiguous substring.
- A condition must remain true while the right boundary expands.
- When a duplicate appears, the left boundary must move forward.
- A character-to-index map allows the invalid prefix to be skipped.

Brute-force reasoning

Start every substring position, expand until a duplicate appears, and track the maximum valid length.

- Time: `O(n²)` with a set-based expansion.
- Space: `O(min(n, alphabet_size))`.

Optimized invariant

At the beginning of every iteration, the window `s[left:right]` contains no duplicate characters.

For the current character:

- If its most recent position is inside the current window, move `left` to one position after that occurrence.
- Record the current index.
- Update the maximum window length.

The left pointer must never move backward.

Python solution

```
def length_of_longest_substring(s: str) -> int:
    """Return the longest substring length with no repeated characters."""
    left = 0
    best = 0
    last_seen: dict[str, int] = {}

    for right, char in enumerate(s):
        previous_index = last_seen.get(char)

        if previous_index is not None and previous_index >= left:
            left = previous_index + 1

        last_seen[char] = right
        best = max(best, right - left + 1)

    return best
```

Edge cases

```
assert length_of_longest_substring("") == 0
assert length_of_longest_substring("a") == 1
assert length_of_longest_substring("aaaa") == 1
assert length_of_longest_substring("abba") == 2
assert length_of_longest_substring("abcabcbb") == 3
assert length_of_longest_substring("pwwkew") == 3
assert length_of_longest_substring(" ") == 1
```

The `"abba"` test catches the common bug where `left` is incorrectly moved backwards.

Complexity

- Time: `O(n)`.
- Space: `O(min(n, alphabet_size))`.

Coding evaluation

Result	Assessment
Correct optimal solution in ≤20 minutes	Strong
Correct optimal solution in 21–30 minutes	Acceptable
Correct brute force, incomplete optimization	Weak recognition
Incorrect handling of `"abba"`	Window invariant not understood
Correct code but cannot explain invariant	Not yet senior-level

---

11. Refactor one high-risk PoC component

Because the actual PoC source code was not supplied, I cannot honestly identify its weakest implemented component. The component that deserves the strictest review is **idempotent ingestion**, because mistakes there directly corrupt financial results.

Required database constraints

```
CREATE TABLE ingestion_requests (
    idempotency_key TEXT PRIMARY KEY,
    payload_hash    TEXT NOT NULL,
    status          TEXT NOT NULL
        CHECK (status IN ('PROCESSING', 'COMPLETED')),
    response_body   JSONB,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at    TIMESTAMPTZ
);

CREATE UNIQUE INDEX expenses_source_record_id_uq
    ON expenses (source_record_id);
```

Refactored service flow

```
from __future__ import annotations

from dataclasses import dataclass
from hashlib import sha256
import json
from typing import Protocol, Sequence


class IdempotencyConflict(Exception):
    pass


@dataclass(frozen=True)
class ExpenseInput:
    source_record_id: str
    department_id: int
    cost_center_id: int
    vendor_id: int
    period: str
    amount: str
    approval_status: str


@dataclass(frozen=True)
class IngestionResult:
    request_id: str
    inserted: int
    updated: int
```

```
class IngestionRepository(Protocol):
    async def claim_request(
        self,
        idempotency_key: str,
        payload_hash: str,
    ) -> bool:
        """Atomically insert the request record if it does not exist."""

    async def get_request(self, idempotency_key: str):
        ...

    async def upsert_expenses(
        self,
        rows: Sequence[ExpenseInput],
    ) -> tuple[int, int]:
        ...

    async def complete_request(
        self,
        idempotency_key: str,
        result: IngestionResult,
    ) -> None:
        ...

    def transaction(self):
        ...


class CacheVersionStore(Protocol):
    async def increment(self, namespace: str) -> None:
        ...


def canonical_payload_hash(rows: Sequence[ExpenseInput]) -> str:
    canonical_rows = [
        {
            "source_record_id": row.source_record_id,
            "department_id": row.department_id,
            "cost_center_id": row.cost_center_id,
            "vendor_id": row.vendor_id,
            "period": row.period,
            "amount": row.amount,
            "approval_status": row.approval_status,
        }
        for row in rows
    ]

    encoded = json.dumps(
        canonical_rows,
        sort_keys=True,
        separators=(",", ":"),
    ).encode("utf-8")

    return sha256(encoded).hexdigest()
```

```
async def ingest_expenses(
    *,
    idempotency_key: str,
    rows: Sequence[ExpenseInput],
    repository: IngestionRepository,
    cache_versions: CacheVersionStore,
) -> IngestionResult:
    if not rows:
        raise ValueError("At least one expense row is required")

    payload_hash = canonical_payload_hash(rows)

    async with repository.transaction():
        claimed = await repository.claim_request(
            idempotency_key=idempotency_key,
            payload_hash=payload_hash,
        )

        if not claimed:
            existing = await repository.get_request(idempotency_key)

            if existing.payload_hash != payload_hash:
                raise IdempotencyConflict(
                    "The idempotency key was used with a different payload"
                )

            if existing.status == "COMPLETED":
                return IngestionResult(**existing.response_body)

            raise RuntimeError("An identical request is already processing")

        inserted, updated = await repository.upsert_expenses(rows)

        result = IngestionResult(
            request_id=idempotency_key,
            inserted=inserted,
            updated=updated,
        )

        await repository.complete_request(idempotency_key, result)

    # This occurs only after PostgreSQL has committed.
    # Cache failure must not roll back the committed financial mutation.
    await cache_versions.increment("finance-analytics")

    return result
```

Important correctness conditions

- `claim_request` must use one atomic `INSERT ... ON CONFLICT DO NOTHING`.
- The payload hash must use a canonical representation.

- Expense uniqueness must also be protected by a database constraint.
- The ingestion request and expense mutation must share one transaction.
- Cache invalidation occurs only after commit.
- A Redis failure does not make the committed PostgreSQL transaction disappear.
- Logging should use a hash or truncated representation of the idempotency key rather than exposing sensitive values.

---

12. Missing tests to add

P0 tests

Test	Failure caught
Same key and same payload submitted twice	Duplicate side effects
Same key and different payload	Incorrect key reuse
Two concurrent requests using the same key	Race condition
Same source record in separate ingestion batches	Duplicate financial record
Failure halfway through batch	Partial transaction
Cache invalidation attempted before commit	Stale or inconsistent cache
Redis unavailable after database commit	Incorrect rollback or false failure
Zero budget	Division-by-zero or misleading percentage
Equal variance values	Unstable ranking and pagination
New row inserted between cursor pages	Missing or duplicated records

Representative concurrency test

```
import asyncio
import pytest


@pytest.mark.asyncio
async def test_concurrent_replay_creates_one_financial_effect(
    ingestion_service,
    expense_payload,
    expense_repository,
):
    results = await asyncio.gather(
        ingestion_service.ingest(
            idempotency_key="request-123",
            rows=expense_payload,
        ),
        ingestion_service.ingest(
            idempotency_key="request-123",
            rows=expense_payload,
        ),
        return_exceptions=True,
```

```
    )

    successful_results = [
        result for result in results
        if not isinstance(result, Exception)
    ]

    assert len(successful_results) >= 1
    assert await expense_repository.count_by_source_id(
        expense_payload[0].source_record_id
    ) == 1
```

The final expected response policy—replay, `202`, or temporary conflict—must be explicitly defined. The essential invariant is that the financial mutation occurs once.

---

13. Reproducibility review

A clean setup should require only documented commands similar to:

```
git clone <repository>
cd <repository>

cp .env.example .env

docker compose up -d --build

alembic upgrade head

python -m scripts.generate_finance_data \
  --seed 7 \
  --output data/finance.csv

python -m scripts.ingest_file \
  --file data/finance.csv \
  --idempotency-key clean-setup-seed-7

pytest -q
```

Reproducibility requirements

- Dependency versions are locked.
- `.env.example` documents every required variable.
- Database migrations start from an empty database.
- Seed generation accepts a fixed seed.
- Generated IDs and dates are deterministic where expected.
- Tests do not depend on execution order.
- Tests do not call real external systems.
- PostgreSQL and Redis test services are isolated.
- API examples in the README are executable.

- Pagination includes a unique tie-breaker.
- Timezone behavior is explicit.
- No developer-specific filesystem path is required.
- One command or CI job runs formatting, type checking, linting, and tests.

Evidence currently available

The PoC implementation, repository, test output, coverage output, migration history, and clean-setup logs were not included in this conversation. Therefore:

- Correctness: **unverified**
- Reproducibility: **unverified**
- Test completeness: **unverified**
- API behavior: **unverified**
- Statistical calculation: **unverified**

Calling these areas complete without repository evidence would be an invented claim.

---

14. Three-minute PoC demo structure

0:00–0:30 — Problem

> The service analyses budget-versus-actual performance and identifies expense exceptions across departments, cost centres, vendors, and accounting periods.

0:30–1:00 — Architecture

> FastAPI exposes ingestion and analytical endpoints. PostgreSQL is the system of record. Redis caches aggregate reads. Ingestion uses idempotency keys and database uniqueness constraints. Structured logs carry correlation IDs.

1:00–1:40 — Demonstration

Show:

1. One ingestion request.
2. Replay of the same request without duplicate records.
3. Variance summary.
4. Top exception query.
5. Cursor-based drill-down.

1:40–2:20 — Correctness and reliability

> Budgets and expenses are aggregated to the same grain before joining. Database writes are transactional. Stable pagination uses a unique tie-breaker. Redis failure falls back to PostgreSQL. Tests cover duplicates, rollback, zero budgets, and deterministic ordering.

2:20–2:50 — Statistical result

> The service reports both the estimated financial effect and its uncertainty. Statistical significance is not treated as sufficient evidence of business value.

2:50–3:00 — Trade-off

> The current design is optimized for an interview-grade PoC. At larger ingestion volume, I would separate ingestion into asynchronous jobs and introduce incremental summary tables or controlled materialized views.

Demo failure condition

If you spend more than 45 seconds listing technologies, the demo is too implementation-focused.

---

15. Concise interview answers

Why PostgreSQL?

> The data is relational, financially sensitive, and requires constraints, transactions, analytical SQL, and auditability. PostgreSQL provides those capabilities while keeping the PoC operationally simple.

Why Redis?

> Redis reduces repeated aggregate-query latency. It is an optimization only; PostgreSQL remains the source of truth, and the service can degrade to direct database reads.

How do you guarantee idempotency?

> I atomically associate an idempotency key with a canonical payload hash, persist the completed response, and protect each upstream record with a unique database constraint. Replays return the stored result, while key reuse with a different payload is rejected.

Why cursor pagination?

> Cursor pagination avoids large offset scans and remains stable under insertion when ordering uses immutable columns and a unique tie-breaker.

Why async?

> I use async only where requests spend substantial time waiting on async-compatible I/O. Blocking libraries are not placed directly inside the event loop.

What does the confidence interval add?

> It communicates the uncertainty around the estimated financial impact. A point estimate alone can create false precision.

---

16. Weak-area recovery register

Populate this using actual quiz and mock results.

Weak area	Evidence of weakness	Required recovery exercise	Exit criterion
Async boundaries	Could not identify blocking calls	Convert one mixed sync/async request path and test concurrency	Correctly explain event-loop blocking and choose a suitable execution model
Idempotency	Relied only on client key	Implement payload hash, atomic claim, response replay, and uniqueness constraint	Concurrent replay creates one business effect
Stable pagination	Used offset or non-unique sort	Implement `(sort_value, primary_key)` cursor	No duplicate or missing rows in insertion test
Analytical SQL	Joined facts before aggregation	Rewrite query using equal-grain CTEs	Correct totals for one-to-many data
Index reasoning	Listed indexes without plans	Run and explain `EXPLAIN (ANALYZE, BUFFERS)`	Can identify scan, sort, join, and cardinality issues
Redis consistency	Invalidated before commit	Move invalidation after commit and add fallback	Database result remains correct when Redis fails
Testing	Tested only status codes	Add state, rollback, race, and deterministic-order assertions	P0 failure modes have executable tests
Statistics	Equated significance with importance	Practise two executive summaries	Includes effect, uncertainty, limitation, and action
Verbal design	Started with tools	Repeat 30-minute mock using requirements-first structure	Completes design with five minutes for trade-offs
DSA	Could not state invariant	Solve three sliding-window problems aloud	Correct invariant before coding

---

17. Week 1 scorecard

Evidence-based status

Category	Weight	Current evidence-based status
Python architecture and code separation	10	Not assessed from implementation
FastAPI contracts and validation	10	Not assessed from implementation
PostgreSQL schema and analytical SQL	15	Conceptual prompts covered; implementation unverified
Redis caching and degradation	10	Conceptual prompts covered; implementation unverified

Async, timeout, retry, and cancellation	10	Conceptual prompts covered; implementation unverified
Testing and failure modes	15	Test requirements defined; execution unverified
Logging and observability	10	Conceptual requirements defined; logs unverified
Statistics and experimentation	10	Conceptual coverage present; verbal performance unassessed
DSA coding accuracy	5	Not assessed until timed attempt
Communication and demo quality	5	Not assessed until recorded or live attempt
Total	**100**	**No honest numerical score can yet be assigned**

Complete this after the exercises

```
Quiz:                  __ / 20
Design mock:           __ / 40
SQL mock:              Pass / Partial / Fail
Statistics response:   Strong / Acceptable / Weak
DSA:
  Correctness:         Pass / Fail
  Time:                __ minutes
  Complexity:          Optimal / Non-optimal
PoC clean setup:       Pass / Fail / Not attempted
PoC full tests:        Pass / Fail / Not attempted
Three-minute demo:     Pass / Over time / Incomplete
```

Week 1 readiness decision

- **Ready to progress:** quiz ≥15, design ≥30, SQL correct or nearly correct, timed DSA completed, and clean setup passes.
- **Progress with recovery tasks:** one area below threshold but no P0 PoC correctness failure.
- **Repeat consolidation:** ingestion can duplicate data, transactions are unsafe, SQL totals are wrong, or the project cannot be reproduced from a clean environment.

Next-week priorities

1. Obtain executable proof of clean setup and deterministic tests.
2. Close any idempotency, transaction, pagination, or aggregation correctness gaps before adding new features.
3. Practise one requirements-first design explanation and one analytical SQL problem daily.
4. Record the three-minute PoC demonstration and remove unnecessary technology narration.
5. Convert every quiz mistake into one executable example or test rather than rereading notes.

Day 7 DSA Add-on — Queue

Revision summary

A **queue** follows FIFO: the first element inserted is the first removed.

In Python, use `collections.deque` for ordinary queue operations:

```
from collections import deque

queue = deque()

queue.append("A")
queue.append("B")

first = queue.popleft()  # "A"
```

Do not use `list.pop(0)` for a queue because removing the first list element is `O(n)`.

Queue concepts

Concept	Typical operation	Python choice	Common use
FIFO queue	Add at rear, remove from front	`deque.append`, `deque.popleft`	Scheduling, buffering
Deque	Add/remove at either end	`collections.deque`	Sliding windows, BFS
BFS	Process nodes level by level	`deque`	Graphs, grids, shortest unweighted path
Producer-consumer	Producers enqueue; consumers dequeue	`queue.Queue` or `asyncio.Queue`	Workers, ingestion pipelines
Bounded queue	Maximum capacity	`Queue(maxsize=n)`	Backpressure and overload control

Producer-consumer essentials

A production producer-consumer system normally needs:

- A bounded queue to prevent unlimited memory growth.
- Multiple consumers only when operations are safe to run concurrently.
- A shutdown mechanism, often a sentinel value.
- Error handling so one failed task does not terminate the worker unexpectedly.
- `task_done()` and `join()` when completion tracking is required.
- Idempotency when processing a message more than once would be harmful.

For threaded Python workers, use `queue.Queue`. For asynchronous workers, use `asyncio.Queue`.

Go provides a similar model through channels. A buffered channel behaves like a bounded queue and naturally introduces backpressure when full.

Recognition signals

A queue is likely appropriate when the problem contains one or more of these signals:

1. Items must be processed in arrival order.
2. The problem asks for minimum steps, minimum time, or shortest distance in an unweighted graph.
3. Processing happens level by level.
4. A change spreads simultaneously from multiple starting points.
5. You need to explore all immediate neighbours before moving farther away.
6. The problem contains workers, jobs, events, messages, or requests waiting to be processed.

Typical BFS language includes:

- “Minimum number of moves”
- “Nearest”
- “Spread”
- “Every minute”
- “Connected cells”
- “Shortest unweighted path”

Timed medium problem — Rotting Oranges

Recommended time: **25 minutes**

You are given an `m × n` grid:

- `0` represents an empty cell.
- `1` represents a fresh orange.
- `2` represents a rotten orange.

Every minute, a fresh orange becomes rotten when it is directly adjacent—up, down, left, or right—to a rotten orange.

Return the minimum number of minutes required for every fresh orange to become rotten. Return `-1` when this is impossible.

Examples

```
Input:
[
  [2, 1, 1],
  [1, 1, 0],
  [0, 1, 1]
]

Output: 4
```

```
Input:
[
  [2, 1, 1],
  [0, 1, 1],
  [1, 0, 1]
]

Output: -1
```

```
Input:
[
  [0, 2]
]

Output: 0
```

Brute-force reasoning

A direct simulation could scan the entire grid once per minute:

1. Find every orange that is currently rotten.
2. Find adjacent fresh oranges.
3. Mark those fresh oranges to become rotten after the current minute.
4. Repeat until no fresh oranges remain or no changes occur.

The important detail is that newly rotten oranges must not spread again during the same minute.

Brute-force complexity

Let:

```
N = rows × columns
```

A complete grid scan costs $O(N)$.

In the worst case, rotting may advance only one cell per minute and require $O(N)$ minutes.

Therefore:

- Time: $O(N^2)$
- Space: $O(N)$ if newly rotten cells are temporarily stored

The repeated scans are unnecessary because only the newly rotten frontier matters.

Optimized reasoning — Multi-source BFS

All initially rotten oranges begin spreading at the same time.

Therefore, instead of starting BFS from one rotten orange, place **every initially rotten orange** into the queue.

This is multi-source BFS.

Algorithm

1. Traverse the grid.
2. Add all rotten-orange positions to the queue.
3. Count all fresh oranges.
4. Process the queue level by level.
5. Each BFS level represents one minute.
6. When a fresh neighbour becomes rotten:
    - Change it to `2`.
    - Decrement the fresh-orange count.
    - Add it to the queue.
7. Stop when:
    - All fresh oranges have rotted, or
    - The queue becomes empty.

Why BFS is correct

BFS processes cells in increasing order of distance from the initial rotten oranges.

Because every edge between adjacent cells takes exactly one minute, the first time a fresh orange is reached is the earliest minute at which it can become rotten.

Starting from all rotten oranges simultaneously ensures that each cell is reached from its nearest initial rotten source.

Key invariant

At the beginning of each BFS level:

> Every position currently in the queue represents an orange that became rotten at the same minute.

After processing that level, all newly enqueued oranges represent the next minute.

Python solution

```
from collections import deque
from typing import List


def oranges_rotting(grid: List[List[int]]) -> int:
    """
    Return the minimum number of minutes required to rot all fresh oranges.
```

```
    Time complexity:
        O(rows * columns)

    Space complexity:
        O(rows * columns)
    """
    if not grid or not grid[0]:
        return 0

    rows = len(grid)
    columns = len(grid[0])

    queue: deque[tuple[int, int]] = deque()
    fresh_count = 0

    # Collect every initial BFS source and count fresh oranges.
    for row in range(rows):
        for column in range(columns):
            if grid[row][column] == 2:
                queue.append((row, column))
            elif grid[row][column] == 1:
                fresh_count += 1

    # Nothing needs to spread.
    if fresh_count == 0:
        return 0

    directions = (
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1),
    )

    minutes = 0

    while queue and fresh_count > 0:
        current_level_size = len(queue)

        # Every item in this level spreads during the same minute.
        for _ in range(current_level_size):
            row, column = queue.popleft()

            for row_delta, column_delta in directions:
                next_row = row + row_delta
                next_column = column + column_delta

                inside_grid = (
                    0 <= next_row < rows
                    and 0 <= next_column < columns
                )

                if not inside_grid:
```

```
                    continue

                if grid[next_row][next_column] != 1:
                    continue

                # Mark immediately to prevent duplicate queue insertion.
                grid[next_row][next_column] = 2
                fresh_count -= 1
                queue.append((next_row, next_column))

        minutes += 1

    return minutes if fresh_count == 0 else -1
```

Walkthrough

For:

```
2 1 1
1 1 0
0 1 1
```

Initial state:

```
Queue: [(0, 0)]
Fresh: 6
Minutes: 0
```

After minute 1:

```
2 2 1
2 1 0
0 1 1
```

After minute 2:

```
2 2 2
2 2 0
0 1 1
```

After minute 3:

```
2 2 2
2 2 0
0 2 1
```

After minute 4:

```
2 2 2
2 2 0
0 2 2
```

All fresh oranges are rotten, so the answer is `4`.

Edge cases

1. Empty grid

```
assert oranges_rotting([]) == 0
```

2. No fresh oranges

```
grid = [[0, 2]]
assert oranges_rotting(grid) == 0
```

3. Fresh oranges but no rotten source

```
grid = [
    [1, 1],
    [1, 1],
]

assert oranges_rotting(grid) == -1
```

4. Isolated fresh orange

```
grid = [
    [2, 0, 1],
]

assert oranges_rotting(grid) == -1
```

5. Multiple initial rotten oranges

```
grid = [
    [2, 1, 1],
    [1, 1, 1],
    [1, 1, 2],
]

assert oranges_rotting(grid) == 2
```

6. Single fresh orange

```
grid = [[1]]
assert oranges_rotting(grid) == -1
```

7. Single rotten orange

```
grid = [[2]]
assert oranges_rotting(grid) == 0
```

Complexity

Let:

```
rows = m
columns = n
```

Each grid cell is examined during the initial traversal.

Each fresh orange is added to the queue at most once and removed at most once.

Therefore:

- Time: `O(m × n)`
- Space: `O(m × n)` in the worst case

Common mistakes

1. Starting BFS from only one rotten orange

All initially rotten oranges spread simultaneously. The queue must initially contain all of them.

2. Using `list.pop(0)`

This costs `O(n)` per removal. Use `deque.popleft()`.

3. Marking a cell only when it is removed

The same fresh cell could be added by multiple rotten neighbours. Mark it rotten immediately when enqueuing it.

4. Incrementing minutes for every orange

Minutes correspond to BFS levels, not queue elements.

5. Returning `minutes - 1` without a clear invariant

That adjustment is often used to compensate for processing an unnecessary final level. It is safer to increment time only while fresh oranges remain.

6. Forgetting unreachable fresh oranges

After BFS ends, inspect `fresh_count`. A positive count means the answer is `-1`.

Concise interview explanation

> This is a multi-source BFS problem because all initially rotten oranges spread simultaneously and every move to an adjacent cell has equal cost. I enqueue every rotten orange, count the fresh oranges, and process the queue level by level. Each level represents one minute. A fresh orange is marked rotten when it is enqueued so it cannot be inserted twice. If fresh oranges remain after the queue is exhausted, they are unreachable. The solution takes `O(mn)` time and `O(mn)` space.

Self-review score

Criterion	Points
Recognized multi-source BFS	2
Initialized all rotten oranges	2
Maintained fresh count	1
Processed level by level	2
Marked cells when enqueued	1
Handled impossible case	1
Correct complexity explanation	1
Total	**10**

A strict target is **8/10 or higher within 25 minutes**.